

Лабораторная работа для 2ИБ

Исследование и реализация системы резервного копирования данных

Дисциплина: Информационные технологии

Тема: резервное копирование, восстановление данных, журналирование операций

1. Цель работы

Изучить основные подходы к резервному копированию данных и реализовать простую программу на Python, которая умеет создавать резервные копии выбранной папки, хранить журнал операций и восстанавливать файлы из архива.

2. Что студент должен получить в результате

- папку проекта с Python-программой;
- несколько созданных резервных копий в формате ZIP;
- журнал операций backup_log.txt;
- краткий отчет с описанием алгоритма и скриншотами;
- ответы на контрольные вопросы.

3. Теоретическая справка

- Полное резервное копирование - копируются все выбранные файлы.
- Инкрементальное резервное копирование - копируются только файлы, изменившиеся после предыдущей копии.
- Дифференциальное резервное копирование - копируются файлы, изменившиеся после последней полной копии.
- Восстановление данных - обратная операция: извлечение файлов из резервной копии.
- Журналирование - запись информации о действиях программы: дата, путь, результат, ошибки.

4. Постановка задачи

Необходимо разработать консольную программу, которая выполняет следующие функции:

1. Создает полную резервную копию выбранной папки.
2. Сохраняет резервную копию в отдельную папку backups.
3. Добавляет в имя архива дату и время создания.
4. Ведет журнал операций в файле backup_log.txt.

5. Показывает список доступных резервных копий.
6. Восстанавливает данные из выбранной резервной копии.
7. Корректно обрабатывает ошибки: отсутствие папки, отсутствие архивов, неверный ввод.

5. Структура проекта

```
backup_lab/
├── main.py
├── backup_log.txt          # создается автоматически
├── data/                  # папка с тестовыми файлами
│   ├── text1.txt
│   └── text2.txt
└── backups/              # сюда будут сохраняться ZIP-архивы
```

6. Пошаговое выполнение работы

Шаг 1. Создать папку проекта

Создайте папку `backup_lab`. Внутри нее создайте папку `data`. В папку `data` поместите несколько тестовых файлов: например `text1.txt`, `text2.txt`, `image.png`.

Шаг 2. Создать файл `main.py`

Откройте проект в VS Code или PyCharm. Создайте файл `main.py`. В нем будет размещена программа.

Шаг 3. Подключить библиотеки

Для работы понадобятся только стандартные библиотеки Python: `os`, `zipfile`, `datetime`, `shutil`.

Шаг 4. Реализовать функцию журналирования

Каждая важная операция должна записываться в файл `backup_log.txt`.

Шаг 5. Реализовать создание резервной копии

Программа должна пройти по всем файлам в исходной папке и добавить их в ZIP-архив.

Шаг 6. Реализовать просмотр резервных копий

Пользователь должен видеть список архивов, которые можно восстановить.

Шаг 7. Реализовать восстановление

Выбранный архив должен распаковываться в отдельную папку `restored_data`.

Шаг 8. Проверить программу

Удалите или измените тестовый файл, затем восстановите данные из резервной копии и сравните результат.

Лабораторная работа 2ИБ. Резервное копирование данных

Шаг 9. Подготовить отчет

В отчет включите цель, описание алгоритма, листинг кода, скриншоты запуска, выводы и ответы на вопросы.

7. Полный пример программы

```
import os
import zipfile
import shutil
from datetime import datetime

BACKUP_DIR = "backups"
LOG_FILE = "backup_log.txt"
RESTORE_DIR = "restored_data"

def write_log(message: str) -> None:
    """Записывает сообщение в журнал операций."""
    time_now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG_FILE, "a", encoding="utf-8") as file:
        file.write(f"[{time_now}] {message}\n")

def create_backup(source_folder: str) -> None:
    """Создает ZIP-архив с полной резервной копией папки."""
    if not os.path.exists(source_folder):
        print("Ошибка: исходная папка не найдена.")
        write_log(f"Ошибка создания копии: папка {source_folder} не найдена")
        return

    os.makedirs(BACKUP_DIR, exist_ok=True)

    time_mark = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    archive_name = f"backup_{time_mark}.zip"
    archive_path = os.path.join(BACKUP_DIR, archive_name)

    files_count = 0

    with zipfile.ZipFile(archive_path, "w", zipfile.ZIP_DEFLATED) as archive:
        for root, dirs, files in os.walk(source_folder):
            for file_name in files:
                full_path = os.path.join(root, file_name)
                relative_path = os.path.relpath(full_path, source_folder)
                archive.write(full_path, relative_path)
                files_count += 1

    print(f"Резервная копия создана: {archive_path}")
    print(f"Количество файлов: {files_count}")
    write_log(f"Создана резервная копия {archive_path}, файлов: {files_count}")

def list_backups() -> list[str]:
    """Возвращает список доступных ZIP-архивов."""
    if not os.path.exists(BACKUP_DIR):
        return []

    backups = []
    for file_name in os.listdir(BACKUP_DIR):
```

```

        if file_name.endswith(".zip"):
            backups.append(file_name)

    backups.sort()
    return backups

def show_backups() -> None:
    """Показывает список резервных копий на экране."""
    backups = list_backups()

    if not backups:
        print("Резервные копии не найдены.")
        return

    print("Доступные резервные копии:")
    for index, backup in enumerate(backups, start=1):
        print(f"{index}. {backup}")

def restore_backup() -> None:
    """Восстанавливает данные из выбранного ZIP-архива."""
    backups = list_backups()

    if not backups:
        print("Нет резервных копий для восстановления.")
        write_log("Ошибка восстановления: резервные копии не найдены")
        return

    show_backups()

    try:
        choice = int(input("Введите номер копии для восстановления: "))
    except ValueError:
        print("Ошибка: нужно ввести число.")
        return

    if choice < 1 or choice > len(backups):
        print("Ошибка: неверный номер резервной копии.")
        return

    selected_backup = backups[choice - 1]
    archive_path = os.path.join(BACKUP_DIR, selected_backup)

    if os.path.exists(RESTORE_DIR):
        shutil.rmtree(RESTORE_DIR)

    os.makedirs(RESTORE_DIR, exist_ok=True)

    with zipfile.ZipFile(archive_path, "r") as archive:
        archive.extractall(RESTORE_DIR)

    print(f"Данные восстановлены в папку {RESTORE_DIR}")
    write_log(f"Выполнено восстановление из {archive_path} в {RESTORE_DIR}")

def main() -> None:
    """Главное меню программы."""
    while True:
        print("\n=== Система резервного копирования ===")

```

```

print("1. Создать резервную копию")
print("2. Показать список резервных копий")
print("3. Восстановить данные")
print("4. Выход")

command = input("Выберите действие: ")

if command == "1":
    folder = input("Введите путь к папке для копирования: ")
    create_backup(folder)
elif command == "2":
    show_backups()
elif command == "3":
    restore_backup()
elif command == "4":
    print("Работа завершена.")
    break
else:
    print("Неизвестная команда. Повторите ввод.")

if __name__ == "__main__":
    main()

```

8. Как запустить программу

1. Откройте терминал в папке backup_lab.
2. Проверьте, что установлен Python: `python --version` или `python3 --version`.
3. Запустите программу командой: `python main.py` или `python3 main.py`.
4. Выберите пункт 1 и введите путь data.
5. Выберите пункт 2 и убедитесь, что архив появился в списке.
6. Измените или удалите файл из data.
7. Выберите пункт 3 и восстановите данные.
8. Проверьте папку restored_data.

9. Пример работы программы

```

=== Система резервного копирования ===
1. Создать резервную копию
2. Показать список резервных копий
3. Восстановить данные
4. Выход
Выберите действие: 1
Введите путь к папке для копирования: data
Резервная копия создана: backups/backup_2026-06-04_12-30-15.zip
Количество файлов: 2

```

10. Дополнительное задание повышенной сложности

Реализуйте инкрементальное резервное копирование. Для этого можно хранить сведения о последней дате изменения каждого файла в отдельном файле `manifest.json`.

- Если файл новый - добавить его в архив.
- Если файл изменился - добавить его в архив.
- Если файл не изменился - не копировать повторно.
- Сравнить размер полного и инкрементального архива.

11. Что должно быть в отчете

1. Титульный лист.
2. Тема и цель работы.
3. Краткая теория: виды резервного копирования.
4. Описание алгоритма программы.
5. Структура проекта.
6. Листинг программы.
7. Скриншоты создания и восстановления резервной копии.
8. Таблица тестирования.
9. Вывод.

12. Таблица тестирования

№	Проверяемое действие	Ожидаемый результат	Фактический результат
1	Создание копии папки data	Создан ZIP-архив	
2	Просмотр списка копий	Архив отображается в списке	
3	Восстановление архива	Появляется папка <code>restored_data</code>	
4	Ввод несуществующей папки	Появляется сообщение об ошибке	
5	Проверка <code>backup_log.txt</code>	В журнале есть записи операций	

13. Контрольные вопросы

1. Чем полное резервное копирование отличается от инкрементального?

2. Почему резервные копии желательно хранить отдельно от исходных данных?
3. Для чего нужен журнал операций?
4. Какие ошибки может допустить пользователь при восстановлении данных?
5. Почему важно периодически проверять восстановление из резервной копии?
6. Какие данные не следует хранить в открытом виде?
7. Как можно защитить резервную копию от несанкционированного доступа?

14. Критерии оценивания

Критерий	Баллы	Комментарий
Создание ZIP-архива	2	Архив создается корректно
Восстановление данных	2	Данные извлекаются в отдельную папку
Журналирование	1	Файл backup_log.txt содержит записи
Обработка ошибок	1	Программа не завершается аварийно
Отчет	2	Есть описание, скриншоты, выводы
Ответы на вопросы	2	Ответы полные и осмысленные

15. Вывод

В ходе лабораторной работы студент изучает практическое значение резервного копирования, реализует простую систему защиты данных от случайного удаления и получает навыки работы с файлами, архивами, журналами и обработкой ошибок в Python.